The Ch(e)at Code

Agent Patterns for Copilot Chat

↑ ↑ ↓ ↓ ← → ← → B A START

ISSUE #018 • JULY 28, 2026 • 🧠 CONCEPTUAL PATTERN

The agent needs to call an internal API — a legacy ERP, a custom CRM, a proprietary compliance system. No pre-built connector exists. Graph API doesn't cover it. The project stalls at "we can't reach our data." This is the wall that stops 20% of enterprise Copilot deployments — and it's the 20% where the highest-value, most differentiated use cases live.

This issue's pattern: Custom Connector Patterns — the architectural decisions that determine when, why, and how to build a custom connector that lets your Copilot Studio agent reach data no pre-built connector can touch.

🧩 PATTERN BREAKDOWN

Custom Connector Patterns

Copilot Studio + Custom Connectors + Power Platform + Azure API Management

Why this matters: Pre-built connectors cover 80% of M365 scenarios. The other 20% — where the highest-value, most differentiated use cases live — require custom connectors. CSAs who can architect these own the enterprise expansion conversation. If a customer says "Copilot can't reach our data," the custom connector pattern is how you unblock them. (See also: **Issue #006**, where Pete Bonds built a custom connector for Teams transcript access via Graph API — the canonical example of this pattern in action.)

The pattern has four design decisions:

1. **When custom connector vs. plugin action (Power Automate).** Use a custom connector when the API will be reused across multiple agents or flows, it has a stable OpenAPI spec, and it needs to be governed as a platform asset. Use a plugin action with Power Automate HTTP when it's a one-off integration, the API is unstable or undocumented, or you need complex pre/post-processing logic. The rule of thumb: if a second team would benefit from the same integration, make it a connector.

2. **The OpenAPI specification is the foundation.** The connector is only as good as its OpenAPI spec. Minimum viable spec: one GET (read), one POST (write), clear parameter descriptions, and — critically — *scenario-specific operation descriptions*. Why? Copilot Studio's LLM reads the `summary` and `description` fields on every operation to decide when to call the connector. Write for the LLM, not just for developers.

3. **Authentication patterns define your security boundary.** Four options, each with distinct trade-offs:

AUTHENTICATION DECISION MATRIX

API Key — simplest; internal APIs with low security needs

OAuth 2.0 (auth code) — user-delegated access to SaaS APIs

OAuth 2.0 (client creds) — service-to-service, no user context

Entra ID — Microsoft-ecosystem APIs; managed identity where possible

Key design decision: Per-user auth (the connector sees only the calling user's data) vs. service account (the connector sees everything, and the agent filters). Per-user is more secure and simpler to govern. Service account is faster to build but creates a data leakage risk if the agent's filtering logic isn't airtight. Default to per-user unless the API doesn't support it. (See **Issue #004: In-Boundary Processing** — custom connectors inherit the security boundary of the Power Platform environment.)

4. **Error handling and resilience aren't optional.** What happens when the API is slow? Down? Returns garbage? You need a three-tier response strategy: **retry** (transient errors like 429s and 503s) → **fallback** (cached data or a degraded response) → **escalate** (tell the user what happened and offer alternatives). The critical constraint: the agent has roughly 10 seconds before the user thinks it's broken.

ERROR HANDLING FLOW

```
API call initiated (~10s budget)
  ↓
Transient error? → Retry with backoff (1 attempt)
  ↓
Still failing? → Return cached/degraded response
  ↓
No cache available? → Escalate to user with alternatives
  ↓
Log error for ops visibility (Power Automate)
```

⚠ **Design warning:** Copilot Studio's LLM decides when to invoke a connector based entirely on the operation's `description` field in the OpenAPI spec. A technically accurate but vaguely worded description ("GET /api/employees/{id}") will be invoked inconsistently. A scenario-specific description ("Look up an employee's profile, team, and current project assignments by their employee ID") will be invoked reliably. Write your OpenAPI descriptions for the LLM, not for Swagger UI.

🏆 **WHERE THIS PATTERN LANDS**

- **Manufacturing & supply chain** — connecting Copilot to legacy ERP systems (SAP, Oracle) that have REST APIs but no pre-built Power Platform connector
- **Financial services & insurance** — proprietary compliance and risk APIs that must stay within the organization's security boundary
- **Healthcare & life sciences** — internal clinical trial management systems, patient scheduling APIs, and HIPAA-compliant data sources
- **Government & public sector** — internal case management and citizen services systems with custom APIs behind strict network boundaries
- **Any enterprise** where the most valuable data lives behind APIs that Copilot can't reach today — and one custom connector changes that

Positioning tip: Custom connectors unlock the long tail of enterprise use cases. When a customer says "Copilot is great for M365 data, but what about our internal systems?" — that's your opening. The connector pattern turns a Copilot limitation into an expansion opportunity. CSAs who can build these own the conversation.

⚡ **QUICK TIPS**

✓ **Use Copilot Chat to generate the OpenAPI spec.** Paste the API documentation (or even example curl commands) into Copilot Chat and ask it to generate an OpenAPI 3.0 spec. It gets you 80% of the way there in minutes instead of hours.

✓ **Response schema must match exactly.** If the API returns fields not declared in your OpenAPI spec, they're silently dropped by the connector. Declare every field you need in the response schema — don't assume the connector will pass through undeclared data.

✓ **Front production APIs with Azure API Management.** Point your connector at APIM, not directly at the backend. APIM adds rate limiting, caching, retry policies, and request/response logging — all without changing the underlying API. This is the production-readiness layer that separates a demo from a deployment.

✓ **Cross-reference: Issue #006 (Pete Bonds' transcript connector)** is the canonical example of this pattern — a custom connector for Teams transcript access via Graph API. **Issue #004 (In-Boundary Processing)** explains how custom connectors inherit the security boundary of the Power Platform environment.

Next issue: we build one.

Issue #019 takes these patterns and walks through a complete custom connector build, step by step.

The Ch(e)at Code

AIWF Team • AI Business Solutions

Questions or ideas for the next issue? Reply to this email.

Microsoft Internal • Not for external distribution